

Coflow Scheduling for LLM Training

Paper # 146, 12 pages, 15 pages total

Abstract

Training large language models (LLMs) generates diverse conflicting coflows, entailing coflow scheduling to optimize computation-communication overlap and the overall training time. However, existing coflow schedulers are fundamentally limited by endhost-only scheduling and fail to address contentions across and within coflows, leading to sub-optimal performance. This paper presents Hermod, a near-optimal coflow scheduler that orchestrates all coflow types at cluster level during LLM training. The key insight of Hermod is that coflows can be uniquely characterized by three model factors, namely, microbatch ID, coflow type, and layer ID, enabling optimal scheduling decisions. Inspired by this, Hermod adopts a two-layer strategy: (1) model factor-driven inter-coflow priority scheduling to align with the LLM training DAG, and (2) matching-based intra-coflow scheduling to maximize transmission rates within the highest-priority coflows. Our evaluation showcases remarkable training time reductions, with up to 1.44 $\times$ and 2.07 $\times$ speedups over state-of-the-art schedulers in testbed and simulation, respectively.

1 Introduction

Large language models (LLMs) have shown exceptional performance in AI tasks such as machine translation, code generation, and conversational agents [16, 26, 49]. However, training LLMs is highly demanding due to their complex architectures and massive data requirements. To overcome the compute and memory limits of individual accelerators, LLM training is distributed across tens of thousands of accelerators, utilizing parallelization strategies to enable efficient coordination and scaling across the cluster [28, 35, 53].

Parallelism in LLM training involves diverse collective communications: AllReduce, ReduceScatter, and AllGather for data parallelism (DP) and tensor/sequence parallelism (TP/SP) [34, 52, 55, 61]; AlltoAll for expert parallelism (EP) [40, 42, 43]; and send/recv for pipeline parallelism (PP) [31, 47]. From a network perspective, these collective communications can be represented as coflows [18], which can be optimized by coflow scheduling.

Coflow schedulers optimize job completion time (JCT) by reordering communication tasks to address data dependencies inherent in training, minimize computation blockages, and enhance the overlap between communication and computation. For instance, ByteScheduler [52] improves DP efficiency by overlapping DP coflows across layers, while Lina [40] lowers the cost of EP coflows by prioritizing them over DP coflows during the backward pass.

Despite being promising, existing approaches [40, 52] suffer from several key limitations that hinder optimal scheduling. First, they restrict their scheduling strategies to end-hosts and neglect cluster-level coordination, raising contention in the network core. While fine-grained load-balancing [28, 35, 53] can well balance traffic, they obscure coflow semantics and amplify interference across coflows in the absence of cluster-wide scheduling. Second, they fail to account for PP coflows, which are prevalent in both dense and sparse models and exhibit sequential dependencies, where each PP stage must await the completion of the preceding coflow. We find that incorporating PP coflow scheduling can improve LLM training performance by 1.29 $\times$ (§2.3). Lastly, these methods fail to address flow contentions within coflows, resulting in inefficient communication when the coflows are internally imbalanced.

This leads us to a central question: *Can we design a coflow scheduler that optimally orchestrates all types of coflows at cluster level throughout the LLM training process?*

To answer this question, we systematically analyze the LLM training DAG with coflows (§3) and identify three cases where coflows from different collectives overlap and conflict with each other. A key insight emerges from this analysis: coflows can be uniquely characterized by three critical model factors—microbatch ID, coflow type, and layer ID—which collectively determine their priorities for optimal scheduling. These factors are application-specific and remain constant throughout training, making them easy to convey to the underlying transport layer and switches.

Guided by this insight, we present Hermod (§4), a near-optimal coflow scheduler that schedules all types of coflows at cluster level throughout the LLM training process. Hermod consists of two key components: 1) a model factor-driven inter-coflow priority scheduling mechanism with strict prioritization, and 2) a matching-based intra-coflow scheduling strategy tailored to each coflow type. For inter-coflow scheduling, the prioritization is determined by strictly adhering to the training DAG and quantized according to the above three model factors. We analyze the potential conflicts among these factors and resolve them by developing a holistic priority strategy for coflows. For intra-coflow scheduling, the goal is to minimize coflow completion time (CCT) without degrading overall JCT. We propose strategies for each coflow type, ensuring that flows in the highest-priority coflow are transmitted at maximum rates. While this approach is straightforward for balanced DP and PP coflows, managing imbalanced EP coflows is more challenging due

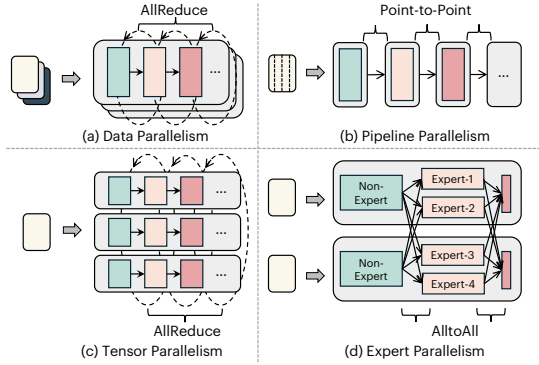


Figure 1: Different parallelisms and their major communication patterns in LLM training.

to varying flow sizes. We leverage framework-level metadata [61] with precise flow size information within EP coflows, and design a matching-based method for EP coflows, leading to optimal performance in LLM training [13, 14].

We have implemented a fully functional prototype of Hermod (§5) and evaluated it using both dense and sparse LLM models through small-scale testbed experiments and large-scale simulations (§6). The results demonstrate that Hermod significantly improves JCT over existing schedulers for all LLM models. Specifically, in the testbed experiments, Hermod improves end-to-end training speed by up to 1.44× in a 32-GPU cluster and 1.38× with 100Gbps network bandwidth, respectively. In the large-scale simulations, the benefits of Hermod are further amplified, yielding speedups of up to 1.50× in a 8192-GPU cluster and 2.07× with a 400Gbps network.

This paper makes the following contributions:

- We conduct a systematic analysis of the LLM training DAG with coflows, identifying three cases of coflow overlaps and three model factors that collectively determine priorities of conflicting coflows (§3).
- We propose a model factor-driven inter-coflow scheduling with strict prioritization, along with a matching-based intra-coflow scheduling tailored to each coflow, to achieve near-optimal communication performance (§4).
- We implement Hermod (§5) and evaluate it on both dense and sparse LLM models, demonstrating that Hermod significantly improves JCT over existing schedulers in both testbed and simulations (§6).

2 Background

2.1 LLM Parallelism

Data parallelism (DP). DP partitions the dataset across devices while replicating model and optimizer states (Figure 1(a)). Each device performs local forward and backward computation, synchronizing gradients via AllReduce [32, 52, 71]. ZeRO [55] improves memory efficiency through fine-

grained state sharding and by decomposing AllReduce into ReduceScatter and AllGather. ZeRO-2 is widely used in LLM training as it reduces memory footprint without additional communication overhead [35, 74]. Various optimizations, including Ring/Tree-AllReduce [5, 34, 56] and hierarchical AllReduce [64], further accelerate synchronization.

Pipeline parallelism (PP). PP partitions the model into stages, each hosted on a different device, reducing per-device memory usage (Figure 1(b)) [31, 47, 48]. Micro-batches flow sequentially through stages, exchanging activations via P2P communication. Techniques such as 1F1B and interleaved 1F1B scheduling alleviate pipeline bubbles [47, 48].

Tensor and sequence parallelism (TP & SP). TP splits tensor operations across devices and aggregates partial results using AllReduce [61] (Figure 1(c)). SP extends TP by partitioning operators (e.g., LayerNorm and Dropout) along the sequence dimension to further reduce memory usage [38]. Together, TP and SP optimize communication by replacing AllReduce with ReduceScatter and AllGather. We use TP to refer to both parallelisms in this paper.

Expert parallelism (EP). EP supports Mixture-of-Experts (MoE) models by distributing expert computations across devices [27, 60]. As shown in Figure 1(d), expert layers rely on AlltoAll communication to dynamically route tokens to experts, with traffic patterns determined by runtime token distributions [33, 40, 42, 43, 69].

Modern LLM training typically combines multiple parallelism strategies to improve efficiency [43, 48]. For example, PTD-P [48] balances computation across GPUs while minimizing pipeline stalls and communication overhead. Due to its high communication cost, TP is usually confined within a single node, whereas DP, PP, and EP commonly scale across multiple nodes [66, 69].

2.2 Coflows in Paralleled LLM Training

Coflow is a classical network abstraction that represents a group of flows sharing a common completion goal [20]. It provides an application-level, holistic view of communication to help avoid stragglers and gain benefits through strategic scheduling and placement [72]. This abstraction has been proven expressive in multiple communication patterns between successive computation stages of data-parallel applications, e.g., MapReduce [24], and Spark [70], etc.

Coflows in LLM training. As LLM training is typically represented as a kind of data-parallel application [48, 52], we summarize the key characteristics of its communication through the coflow perspective in Table 1:

- **DP and TP/SP coflows** use ring- or tree-AllReduce, where flow information—including number, endpoints, size, and distribution—is pre-determined during compilation [12], enabling proactive scheduling and resource allocation.
- **PP coflows** consist of sequential P2P communications be-

Coflow Characteristics		DP	PP	TP & SP	EP
Coflow Type		Ring-based, Tree-based	Point-to-Point	Ring-based, Tree-based	All-to-All
On-Arrival Knowledge	Flow Number	N_{dp}	1	N_{tp}	$N_{ep}(N_{ep} - 1)$
	Flow Endpoints	Known	Known	Known	Known
	Flow Size	Deterministic	Deterministic	Deterministic	On-demand
	Flow Distribution	Balanced	Balanced	Balanced	Imbalanced
Coflow Scope		Inter-rack	Inter-rack	Intra-node	Intra-rack

Table 1: Coflows in different parallelisms.

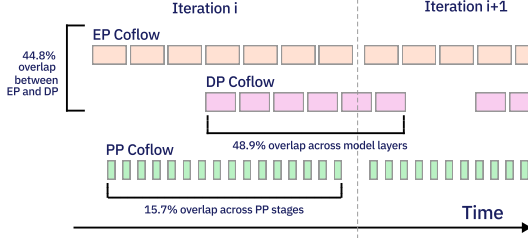


Figure 2: [Production profiling] Cluster-level timeline of EP, PP, and DP coflows for Mixtral-8x7B, showing frequent temporal concurrency among coflows.

tween adjacent pipeline stages. Although each endpoint pair exchanges a single flow, these transfers are still modeled as coflows [20], with flow information known at compile time [12].

- **EP coflows** use AllToAll communication with predefined endpoints and flow counts, while flow sizes are determined dynamically by the gating network based on token routing (§2.1). Skewed token distributions across experts can cause persistent flow imbalance. Prior work shows that this imbalance persists throughout training even with load-balancing losses [27, 42], motivating runtime scheduling to improve the efficiency [40, 42, 43].

The coflow scope aligns with hardware topology, varying in network bandwidth to match the communication volume of each parallelism. DP and PP coflows span racks, TP/SP are intra-node, and EP occur within a single rack [42, 43, 69].

Prevalence of cluster-wide coflow overlap. Based on the above coflow characteristics, we next examine the prevalence of cluster-wide coflow overlap in LLM training to highlight the necessity for optimized coflow scheduling strategies. We use Megatron-LM to profile Mixtral-8x7B [33] on a production-level NVIDIA DGX SuperPOD platform [8], equipped with 128 H-series GPUs (Hopper architecture) connected via 128 400Gbps ConnectX-7 NICs in a rail-optimized topology. All servers are synchronized using Precision-Time Protocol (PTP) [59] for accurate timestamp recording, and the overlap-grad-reduce training parameter is enabled for faster training [9]. By measuring the start and end times of each communication operator, we calculate the overlap duration among them.

Our analysis focuses on inter-node coflows, the primary bottleneck due to the limited 100Gbps RDMA bandwidth compared to the 400Gbps NVLink bandwidth within

nodes [66]. We measure overlap for DP, PP, and EP coflows. Here, "cluster-level" means that we aggregate the timelines of coflows from all workers in the job based on end-host measurements and project them onto a single global time axis. This captures the degree of *temporal overlap* among coflows. Although such concurrency does not imply that these coflows always contend for the same physical links, in a shared network fabric it provides a conservative indicator of the potential for inter-coflow interference at aggregation and core switches.

The results, shown in Figure 2, reveal significant cluster-level coflow overlap¹. Specifically, EP and DP coflows overlap for 44.8% of the iteration time, DP coflows across different layers for 48.9%, and PP coflows across different stages for 15.7%. These frequent overlaps indicate substantial inter-coflow interference potential and lead to degraded communication performance.

2.3 Existing Works on Coflow Scheduling

Several works have been proposed to manage coflow overlaps [29, 32, 40, 52, 71]. Some methods [29, 32, 52, 71] optimize DP by reordering DNN layer transmissions to enhance communication-computation overlap. For example, ByteScheduler [52] prioritizes DP coflows from front layers and uses tensor partitioning to improve overlap. Other approaches [40] focus on EP, prioritizing communication in expert layers during backward propagation to minimize AlltoAll time costs.

Despite their effectiveness, these approaches have several limitations in handling coflow scheduling and addressing contentions across and within coflows:

Limitation #1: Lack of cluster-wide scheduling. Existing approaches [32, 40, 52] primarily focus on scheduling at the endhosts, rather than managing training traffic across the entire cluster. These methods enable coflow preemption only at the point of submission to the communication library (e.g., NCCL [7]), while the underlying transport layer remains unaware of these decisions. As a result, the lack of coordination between the intermediate scheduler and the transport layer can lead to uncoordinated overlap among coflows and suboptimal performance due to inter-

¹Two coflows are considered overlapped when their flows are concurrently in flight at the end hosts.

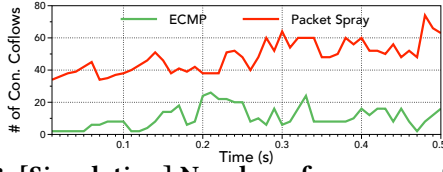


Figure 3: [Simulation] Number of concurrent coflows at a core switch under different load balancing mechanisms.

Configuration	Baseline	DP-first	PP-first	EP-first
Training Speedup	-	0.81×	1.29×	1.51×

Table 2: [Simulation] The potential benefits of scheduling with different prioritization strategies. DP-, PP-, and EP-first are heuristic-based approaches that prioritize DP, PP, and EP traffic, respectively.

ference at core network components, such as aggregate and core switches.

This issue becomes more pronounced in state-of-the-art LLM training clusters that adopt fine-grained load-balancing mechanisms [15, 28, 35, 53]. While such mechanisms help distribute traffic, they also obscure coflow semantics and increase the degree of uncontrolled overlap among coflows in the absence of cluster-wide scheduling. Although we lack permission to capture logs from production switches, we simulate this scenario using the simulator from [67]. We train Mixtral-8×7B [33] in a 1024-GPU cluster using two load balancing mechanisms commonly adopted in recent AI clusters: ECMP [28, 35] and packet spray [15]. By measuring coflow numbers in switch queues every 10μs, we observe that fine-grained packet spray increases concurrent coflows by an average factor of 7.4, as shown in Figure 3. While this metric does not directly measure congestion, it indicates a substantially higher degree of inter-coflow interference potential. Since the transport layer is unaware of coflow priorities, it cannot exploit this information to protect critical coflows at switches, leading to suboptimal performance in LLM training.

Limitation #2: Incomplete consideration of all coflow types. Existing approaches [32, 40, 52] overlook certain coflow types, particularly PP coflows, which are crucial as they typically sit on the critical path of training and can block subsequent stages [31, 47].

We reuse the previous simulation configuration and assess the impact of different parallelism strategies with naive scheduling at the cluster level. The results in Table 2 indicate that prioritizing DP coflows degrades training performance, whereas naive prioritization of PP and EP coflows improves iteration times by 1.29× and 1.51×, respectively. Notably, EP coflows are specific to sparse MoE models, while PP coflows occur in both sparse and dense model training. These findings highlight the need for optimized scheduling across all coflow types—EP, DP, and PP—at the cluster-wide level to

improve LLM training efficiency.

As LLM models scale, PP becomes increasingly vital due to its lower communication costs compared to TP. Experiments in [48] show that while TP degree caps at 8 due to bandwidth limitations [66], PP degree scales with cluster size, leading to greater communication contention. Addressing these challenges is crucial to maintain training efficiency.

Limitation #3: Unaware of flow contentions within coflows. Existing approaches [32, 40, 52] focus solely on scheduling communication operators (*i.e.*, coflows) without accounting for flow contentions within each coflow. This limitation can lead to suboptimal performance, as internal flow contentions can affect overall job completion time [13, 23]. In EP coflows, for example, imbalanced flow distribution among experts can exacerbate contentions, leading to increased training completion times. We elaborate on these contentions in §4.2.1. These approaches work primarily as intermediate schedulers, failing to incorporate fine-grained, flow-level resource allocation strategies from traditional coflow scheduling [13, 21, 23].

2.4 Motivation and Challenge

The above limitations motivate us of a coflow scheduler that can optimally, if possible, consider all coflow types, and be implemented at the cluster-wide level. Specifically, the coflow scheduler should be:

- **Comprehensive:** It should account for all coflow types, including DP, PP, and EP, to optimize training performance across different parallelism strategies.
- **Fine-grained:** It should address flow contentions within coflows and optimize flow-level resource allocation to minimize JCT.
- **Cluster-wise deployable:** It should be aware of coflow overlap throughout training and be deployable across the entire cluster, managing coflow scheduling at both end-hosts and network core components to reduce contention.
- **Optimal:** The scheduler should aim to optimally minimize the completion time of the entire training.

Challenge: Non-trivial efforts in designing a comprehensive policy. To design a comprehensive coflow scheduler which optimally manages all coflow types, a naive approach is to combine existing strategies for each coflow type for a comprehensive policy. Unfortunately, this approach can lead to inefficiencies, especially with the prevalent optimizations such as ZeRO-2 and 1F1B [35, 48]. For instance, combining ByteScheduler and Lina results in EP coflows always prioritize over DP coflows, while DP coflows are prioritized according to their associated layers. This could lead to sub-optimal performance. When crossing iterations, DP coflows are on the critical path and be needed for AllGather oper-

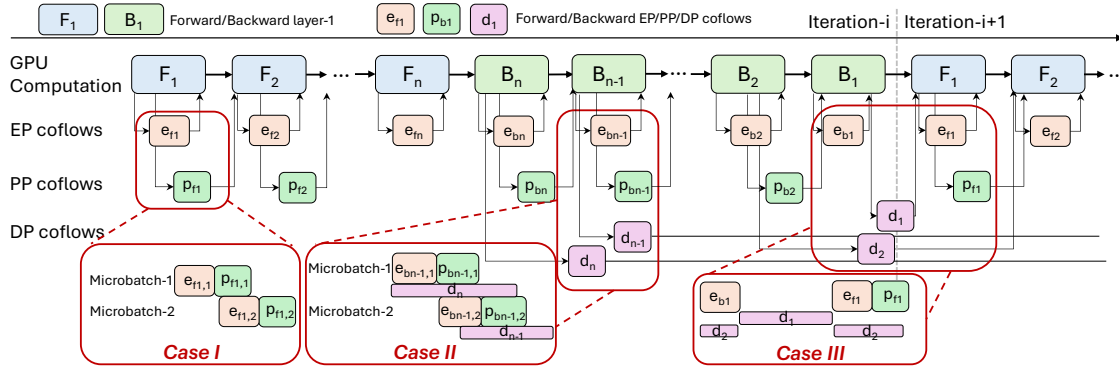


Figure 4: Computation and coflows in LLM training.

ations to gather layer parameters in ZeRO-2 [55]. This results in higher priority of DP coflows over others. Besides, the significance between EP coflows of the front layers and DP coflows of the latter layers is left uninvestigated. Furthermore, pipeline scheduling strategies such as 1F1B introduce an additional microbatch dimension to cluster scheduling, further complicating scheduling decisions—an aspect not yet addressed by existing schedulers.

In summary, existing approaches either require complex combinations to be adaptive with prevalent optimizations or neglect some coflow types with potential performance degradation. To address these challenges, a systematic analysis of coflow interactions regarding the LLM training directed acyclic graph (DAG) is in its urgent need.

3 LLM Training DAG with Coflows

This section critically revisits the LLM training DAG with a focus on coflows, analyzing the overlap among EP, PP, and DP coflows at different training phases. By identifying inefficiencies and proposing a more efficient scheduling policy, we aim to enhance resource utilization and improve overall training performance.

3.1 Problem Statement

We consider the common industry scenario where a dedicated cluster runs a single LLM training job [35]. The network topology, worker placement, and parallelization strategy are predefined before execution, resulting in a fixed LLM training DAG throughout training. This DAG determines coflow dependencies, encompassing all necessary data movements. Formally, it translates into a dependency graph where each coflow represents a set of flows with predefined characteristics, as detailed in Table 1.

The objective is to define a cluster-wide optimal scheduling policy that efficiently allocates network resources among coflows to minimize job completion time (JCT). Specifically, the policy should:

- Adhering to the LLM training DAG: It should strictly respect coflow dependencies as determined by the DAG, in-

corporating conventional mechanisms like ZeRO-2 [55], 1F1B, and its variants [48].

- Minimize global JCT: The policy should minimize JCT of completing the entire coflows. That says, it should avoid coflow overlaps as much as possible, while maximizing communication-computation overlap.
- Cluster-wise deployable: It should be practically deployable across the entire cluster.

3.2 Systematic Analysis of LLM Training DAG

We depict a conventional LLM training DAG in Figure 4, which trains an MoE model consisting of multiple MoE layers with ZeRO-2 and 1F1B optimizations. The training process follows an iterative manner, with phases of forward propagation (FP) and backward propagation (BP) in each iteration. Within each iteration, EP coflows occur twice per MoE layer group for token dispatching and gathering. PP coflows then facilitate intermediate result transmission between layers². These operations are repeated across layers and microbatches. During BP in the current iteration and FP in the next, DP coflows execute as ReduceScatter and AllGather operations for gradient synchronization and parameter updates³ [35, 52, 55, 71].

From the above DAG, we identify in total three cases where coflows may overlap, as highlighted by the red rectangles in Figure 4:

- *Case I: EP and PP coflows among different microbatches.* After processing microbatch-1, the worker invokes the asynchronous operator to send to transmit intermediate results to the next PP stage, generating PP coflows. Simultaneously, it processes microbatch-2, performing attention

²For ease of presentation, we configure each layer to represent a pipeline stage, thus showing PP coflows between layers. In practice, PP coflows occur between stages, which may span multiple layers, depending on the PP degree.

³We use the same layer-wise description as in [52, 71], whereas in PyTorch, DP coflows are grouped into buckets spanning multiple layers while maintaining a sequential execution order [50].

and gate computations before generating EP coflows for token dispatching. Our profiling results in §2.2 show average attention and gate computation times of 2.4ms and 0.6ms, respectively. Given that PP coflows complete in ~4ms on average, there is potential for PP-EP coflow overlap, even at the end host (as we have identified in §2.2). This overlap becomes more likely at the cluster scale, where PP coflows compete for bandwidth with other coflows, prolonging their completion while attention and gate computation times remain constant.

- *Case II: EP, PP, and DP coflows among different microbatches during backward propagation.* Building on *Case I*, DP coflows emerge during backward propagation, overlapping with BP computation. In Megatron-LM, DP coflows are invoked only after the last microbatch completes its gradient calculation for the layer and initiates gradient synchronization. The concurrent nature of DP coflows with computation allows DP to be transmitted simultaneously with other coflows [40].
- *Case III: EP, PP, and DP coflows among different layers during cross-iteration phases.* In this case, DP coflows from latter layers may overlap with EP and PP coflows from front layers during the next iteration's forward propagation. This scenario becomes particularly common when enforcing priority scheduling with tensor partition⁴ [52], as the figure depicted. Unlike *Case II*, DP coflows in this case are critical, as they gather the parameters required to initiate forward propagation in the next iteration [55].

We make a key observation based on the identified cases: cluster-wide coflows throughout training can be uniquely represented by three model factors: microbatch ID (MID), coflow type (CType), and layer ID (LID). Specifically, MID and CType collectively identify coflows in *Case I* and *Case II*, while CType and LID are for coflows in *Case III*.

Notably, these model factors showcase the following characteristics, making them well-suited for optimal coflow scheduling:

- *Pre-determined.* The factors are pre-determined in applications and remain constant throughout training.
- *Simple.* The factors are simple integers that can be easily transformed and transmitted to transport and switches.

4 Hermod

In this section, we present the design of Hermod. We leverage the three model factors and propose a near-optimal inter-coflow priority policy for scheduling all coflow types (§4.1). Building on this strict priority, we further introduce an op-

⁴At the time of writing, Megatron-LM [61] does not yet support priority scheduling. We think this may be partly due to the use of clip-by-norm [54] in LLM training, which can create barriers between iterations. However, clip-by-value [54] could eliminate these barriers algorithmically.

	<i>Case I</i>	<i>Case II</i>	<i>Case III</i>
Priority factor	MicrobatchID (Lower is better)	Coflow type (EP & PP > DP)	LayerID (Lower is better)

Table 3: Priority factors used to assign priority for three cases. The content in the parentheses indicates priority order.

timal intra-coflow scheduling policy to avoid fine-grained flow contentions (§4.2).

4.1 Model factor-driven Inter-Coflow Priority Scheduling

Our goal is to design an optimal inter-coflow scheduling policy to address the identified three cases leveraging three model factors. The primary goal of the coflow scheduling is to minimize the JCT of the training DAG in Figure 4, *i.e.*, reducing the time spent on the critical path (the longest path in DAG where computation/communication *must* be executed) as much as possible and hiding the communication behind computation as much as possible [29, 52, 57].

4.1.1 Model factor-driven priority assignment. We make a key observation based on the above goal: by considering *how many on-critical-path computations can be blocked by the coflows* and *when the transmitted data will be consumed*, we can derive a *near-optimal* priority scheduling policy across all cases. Again, we use the example in Figure 4 for illustration.

- For *Case I*, we must address the contention between $p_{f_{i,1}}$ and $e_{f_{i,2}}$ coflows. We observe that allocating more network resources to $e_{f_{i,2}}$ while delaying $p_{f_{i,1}}$ may accelerate the progress of microbatch-2 in layer-1. However, this progress is stalled until microbatch-1 completes in layer-2, which is delayed due to $p_{f_{i,1}}$, ultimately increasing the overall model processing time. The above phenomenon is due to the sequential nature of microbatch processing, where earlier microbatches block more computations on the critical path. Hence, it is essential to prioritize the coflows of earlier microbatches over the latter ones. This prioritization can be generalized across all coflows within the same iteration.
- For *Case II*, while the priority between EP and PP coflows is determined by the microbatch ID, their relationship with DP coflows must also be considered. Since EP and PP coflows are consumed in the subsequent layer, whereas DP coflows are used in the next iteration, EP and PP coflows should be prioritized over DP coflows. This minimizes blocking time while maximizing the overlap of DP coflows with computation.
- For *Case III*, we analyze the priority between EP/PP coflows from the next iteration and DP coflows from the previous iteration. Given that data from former layers is consumed sooner than that from latter layers, regardless of coflow

Conflicts	MID+CType	MID+LID	CType+LID
Priority factor	/	MID*	/

Table 4: Potential conflicts for different combinations of factors and the corresponding priority factor. / indicates impossible scenario in training, and MID* means that MID is prioritized mostly, unless in certain rare cases (e.g., DP coflows involve in).

type, coflows from earlier layers should be prioritized.

Formally, the above priority decisions are quantified into three model factors, establishing a strict priority order among coflows, as listed in Table 3.

Why near-optimal priority? By definition, the coflow critical path bounds overall JCT [37, 57]. In LLM training DAGs, serving a later-consumed coflow before an earlier one prolongs communication waits and increases idle compute [29, 52]. Our consumption order is near-optimal since: 1) earlier microbatches unlock more compute, 2) EP/PP outputs are consumed within the same iteration while DP extends across iterations, and 3) lower LIDs are consumed earlier. Prioritizing earlier-consumed coflows is conservative in the sense that it does not delay downstream execution, whereas the reverse can stall the critical path.

4.1.2 Handling factor conflicts. While comprehensive the above analysis is, an unresolved question remains: how should coflow priorities be determined when the three model factors conflict? To address this, we rigorously examine all potential conflicts using domain knowledge of LLM training and summarize the resolution in Table 4. Specifically,

- *Conflict between MID and CType:* A potential conflict, such as a DP coflow from an earlier microbatch versus an EP coflow from a later microbatch, does not arise during training. As stated in §3.2, DP coflows are invoked only after the last microbatch has completed its computation. Consequently, DP coflows always carry the highest MID value, inherently avoiding this conflict.
- *Conflict between MID and LID:* A scenario such as a layer-2 coflow from microbatch-1 versus a layer-1 coflow from microbatch-2 can occur in two representative cases. The first case involves EP and PP coflows with different MIDs and LIDs coexisting during FP. Here, we prioritize MID over LID to *minimize the on-critical-path computations' blockages*, ensuring that earlier microbatches are processed with minimal delays. The second case arises during cross-iteration when a layer-1 EP/PP coflow from microbatch-2 overlaps with a layer-2 DP coflow. Since DP coflows always hold the highest MID (e.g., microbatch-4), prioritizing them prevents blocking microbatch-1 in the subsequent iteration (as DP coflows correspond to "microbatch-1" in the next iteration [55]). However, this scenario is exceedingly rare, as DP coflows typically complete be-

fore microbatch-2 begins processing in the next iteration (our measurements indicate that each stage persists for approximately ~100+ ms). Consequently, the general resolution for MID+LID conflicts is: if DP coflows are present, LID takes precedence over MID; otherwise, MID is prioritized. Given the rarity of DP-involved conflicts, the default prioritization strategy remains MID over LID.

- *Conflict between CType and LID:* A case such as a PP coflow from layer-2 versus a DP coflow from layer-1 is infeasible during training. During backward propagation, DP coflow from layer-1 is never generated before PP coflow from layer-2, as DP coflows are always the final coflows produced for a layer. During cross-iteration, the DP coflow from layer-1 *must* be generated before the PP coflow from layer-2 due to dependencies on the corresponding layer in the next iteration. Thus, this conflict does not occur.
- *Conflict among all three factors:* Such a scenario is impossible during training, as demonstrated by the combined analyses above.

In summary, the priority assignment for the three factors follows the order of MID > CType > LID. Notably, the above priority assignment with the three model factors *ultimately* establishes a *strict order* among all coflows in the cluster throughout training.

4.1.3 Priority scheduling in interleaved 1F1B. While our prior analysis assumes the conventional LLM training DAG with the widely used 1F1B pipeline schedule [47, 48], our priority scheduling policy naturally generalizes to other DAG variants, such as interleaved 1F1B [48]. Unlike the conventional schedule, interleaved 1F1B distributes model layers across workers in an interleaved fashion to reduce pipeline bubbles, resulting in interleaved PP coflow transmissions. Our analysis shows that under this variant, PP coflows are no longer on the critical path and can be effectively overlapped with computation, while the prioritization of other coflows remains unchanged. Consequently, only the priority order in *Case II* of Table 3 needs adjustment to EP > PP > DP, where EP coflows remain critical, PP coflows are consumed within the same iteration, and DP coflows span iterations. All other priority assignments in Table 3 remain unchanged. Importantly, the relative ordering of the three factors is preserved, as the underlying conflict cases remain the same. We further validate the effectiveness of Hermod under interleaved 1F1B in our evaluation (§6.2.2).

4.2 Matching-based Optimal Intra-Coflow Scheduling

Given the strict priority of coflows, the next step is to allocate network resources optimally among flows within each coflow to eliminate contention and minimize coflow completion time (CCT).

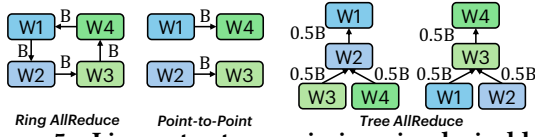


Figure 5: Line-rate transmission is desirable for coflows with balanced flow sizes, e.g., DP & PP coflows.

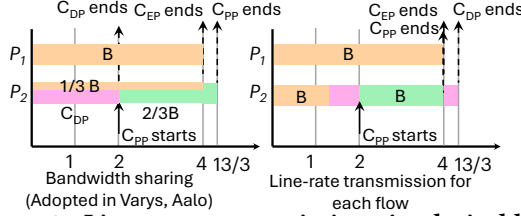


Figure 6: Line-rate transmission is desirable for coflows with imbalanced flow sizes to avoid degrading the global JCT.

4.2.1 Line-rate Transmission is Desirable Following prior work on coflow scheduling [13, 21, 23], effective network resource allocation hinges on optimal rate allocation within each coflow. In this work, we systematically analyze rate allocation strategies tailored to different coflow types. Specifically, we study DP and PP coflows, which exhibit balanced flow distributions, and EP coflows, which are characterized by inherently imbalanced flows, as summarized in Table 1.

Balanced coflows. As Figure 5 shows, balanced coflows exhibit fixed rate allocation strategies that minimize CCT, with flows transmitting at line rate under the optimal schedule. In ring-based DP coflows, this corresponds to forming a ring where each flow transmits at line rate, achieving bandwidth optimality [51]. PP coflows similarly benefit from line-rate transmission, as each coflow contains a single flow. Tree-based DP coflows differ slightly: because two communication trees are required, the available bandwidth is evenly split, with each tree allocated half of the line rate [5].

Imbalanced Coflows. For coflows with imbalanced flow sizes, *i.e.*, EP coflows, the optimal strategy for minimizing CCT is to ensure that the largest flow always transmits at line rate, thereby eliminating intra-coflow contention [13, 21, 23]. Prior work achieves this via two approaches: (i) *bandwidth sharing*, which allocates bandwidth proportional to flow sizes [21, 23], and (ii) *line-rate transmission*, which avoids explicit rate allocation and instead prioritizes flows according to coflow order [13].

In the context of LLM training, we find line-rate transmission more suitable. As illustrated in Figure 6, consider a common scenario during backpropagation where EP (highest priority) and DP (lowest priority) coflows start concurrently, and a PP coflow (second priority) begins at $t = 2$. Under bandwidth sharing, all coflows complete at $\frac{13}{3}$, but the PP coflow finishes later, potentially delaying the global JCT due to its position on the critical path and higher priority over DP coflows. In contrast, with line-rate transmission,

the PP coflow completes alongside the EP coflow while DP coflows transmit in the background, avoiding any delay to the global JCT.

Why line-rate transmission is optimal? Line-rate transmission is optimal over bandwidth sharing in the LLM training context because allocating bandwidth proportionally does not improve the completion time of the highest-priority coflow [13] and can instead negatively impact global JCT, as our example illustrates. In contrast, for an *ordered* coflow, it is better to allocate as much bandwidth as possible to the highest-priority coflow, ensuring all its flows are transmitted quickly and minimizing flow completion times. This principle has been theoretically proved by prior studies: Sincronia demonstrates that the optimal strategy for coflows with strict priority does not require explicit rate allocation and achieves a solution within $2\times$ of the optimal. Similarly, pFabric [14] asserts that, once flow priorities are determined (as in our scenario), the primary role of rate control is to enforce these priorities while preventing network congestion.

4.2.2 Matching, Not Rate Allocation Building on the above rationale, we conclude that the optimal strategy for transmitting imbalanced EP coflows is to transmit all flows at line rate while avoiding network congestion. Consequently, we propose a matching-based scheduling policy for EP coflows to schedule all flows without incurring incast. The basic idea is to divide the transmission phase into multiple sub-phases, with each sub-phase forming a matching within the EP group and transmitting a portion of the flows accordingly.

Challenge and opportunity. A key challenge arises: how can we obtain the flow sizes of EP coflows to be transmitted with minimal overhead? Although the flow sizes for each sender-receiver pair are determined after the gate network computation, naively collecting this information would require additional communication rounds before transmission. As the number of MoE layers increases, such overhead could accumulate and significantly impact training efficiency.

Fortunately, this information is already materialized by the training framework as metadata [6] without any additional overhead. Specifically, to facilitate subsequent AlltoAll communication, frameworks such as Megatron-LM preallocate appropriately sized receiving buffers on GPUs for each sender-receiver pair. Although each worker requires only a subset of the transmission matrix—corresponding to its own send and receive volumes—the framework simplifies the process by performing a full AllGather operation across the EP group. The overhead of this operation is negligible compared to iteration time, *i.e.*, $\sim 0.4\text{ms}$ vs. $\sim 5\text{s}$ based on our logs (§2.2). While we observed this metadata in Megatron-LM [6], a widely used LLM training framework, this practice is necessary in other frameworks as well, as GPU buffer reservation is required prior to transmission. In

Algorithm 1: Finding Matching Sequences and Weights**Input:** Transmission matrix $T \in \mathbb{R}^{n \times n}$ **Output:** Matching sequences $M_1, M_2, \dots, M_k$ and corresponding weights $w_1, w_2, \dots, w_k$

```

1  $matchings \leftarrow \{\}$ ;
2  $weights \leftarrow \{\}$ ;
3  $T \leftarrow array(T)$ ;
4 while  $\exists T_{ij} > 0$  do
5    $row\_ind, col\_ind \leftarrow linear\_sum\_assignment(-T)$ ;
   // Hungarian algorithm
6    $M \leftarrow zeros\_like(T)$ ;
7   for  $i \in row\_ind$  and  $j \in col\_ind$  do
8      $M[i, j] \leftarrow 1$ ;
9    $matching\_values \leftarrow T[row\_ind, col\_ind]$ ;
10   $matching\_weight \leftarrow \min(\{T_{ij} \mid T_{ij} > 0\})$ ;
11   $weights \leftarrow weights \cup \{matching\_weight\}$ ;
12   $matchings \leftarrow matchings \cup \{M\}$ ;
13   $T \leftarrow \max(T - matching\_weight \times M, 0)$ ;
14 return  $matchings, weights$ ;

```

summary, by leveraging this transmission matrix, we can transform flow sizes from unknown to known and design a matching strategy to transmit all flows in the EP coflows without incurring incast.

Matching algorithm. We present our solution for finding matching sequences and weights for EP coflows in Algorithm 1. Our approach leverages the classical Hungarian algorithm [39] to iteratively find the maximum matching in the residual transmission matrix, greedily selecting as many flows as possible for transmission (Line 5). Specifically, the algorithm iteratively computes the maximum matching, subtracts the matched flows from the transmission matrix, and repeats this process until all flows are matched (Line 5-13). The result is a sequence of matching flows and their corresponding weights. Using this sequence, the EP coflow transmission phase is divided into multiple sub-phases, with each sub-phase involving transmitting flow portions proportional to the associated weights by the workers in each matching. Since the input matrix is identical across all workers in the EP group, the derived matching sequences and weights are also identical. Each worker independently runs the same deterministic algorithm locally before transmission, and no runtime coordination or additional network RTT is required. Furthermore, by greedily selecting the maximum matching in the residual matrix, the algorithm maximizes the number of simultaneously scheduled flows in each sub-phase, thereby guaranteeing both the optimal CCT for EP coflows and line-rate transmission for the highest-priority coflows.

5 Implementation

We have implemented a fully functional Hermod prototype in C++ and Python, integrated with Megatron-LM, PyTorch,

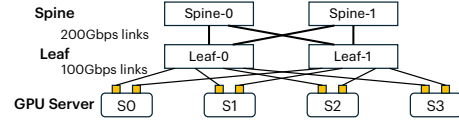


Figure 7: Testbed topology.

and NCCL. The model factor-driven inter-coflow scheduling is enforced at both endhosts and switches, while matching-based intra-coflow scheduling is handled at endhosts using LibTorch.

On each end host, Hermod runs a single-threaded scheduler that maintains a priority queue of coflows, with priorities derived from model factors provided by Megatron-LM. To enforce these priorities in the network, we design a priority-aware networking layer that encodes coflow priorities into the 6-bit DSCP field in a coarse-grained yet deterministic manner. DSCP values are computed independently at each end host using only local information—microbatch ID, coflow type, and layer ID—without global coordination. In our prototype, we use 32 priority levels by bucketizing microbatch ID into four urgency classes, coflow type into two classes (DP vs. EP/PP), and layerID into four classes based on relative model position. The resulting priority (0–31) is directly encoded into the DSCP field, representing a coarse urgency class rather than a global total order; precise ordering among coflows is maintained by the end-host scheduler. Finally, we implemented an asynchronous All to All operator in PyTorch based on our matching algorithm to enforce the proposed transmission logic for EP coflows.

6 Evaluation

We evaluate Hermod’s performance by answering the following questions:

- **How does Hermod perform in end-to-end LLM training?** Testbed experiments (§6.2.1) over three representative LLM models show that Hermod improves the training speed by up to 1.44× over baselines in a 32-GPU cluster, and 1.38× speedup with varying network bandwidths.
- **How does Hermod perform in complex interleaved 1F1B scenario and in terms of its components?** Testbed experiments (§6.2.2) show that Hermod improves training speed by 1.18× in interleaved 1F1B scenario. Additionally, its inter- and intra-coflow scheduling contributes to 1.24× and ~1.11× speedup, respectively.
- **How does Hermod scale with increasing cluster size?** Simulation experiments (§6.3) show that Hermod scales effectively for large-scale model training, outperforming baselines by up to 1.50× in a 8192-GPU cluster, and 2.07× speedup with varying network bandwidths.

6.1 Methodology

Our evaluation comprises both small-scale testbed experiments for moderate-size models and large-scale simulations

Configuration	Models	# of layers	# of experts	PP degree	TP degree	EP degree	Sequence length	Micro-batch size
Testbed	Qwen-MoE-12L	12	64	4	1	8	4096	4
	Mixtral-8×7B	32	8	2	1	8	4096	4
	GPT-3 13B	40	/	4	4	/	2048	4
Simulation	Qwen-MoE-24L	24	64	8	8	32	4096	8
	Mixtral-8×22B	32	8	8	8	8	4096	8
	GPT-3 175B	96	/	8	8	/	2048	8

Table 5: Models and default parallelization strategies used in our testbed and simulation.

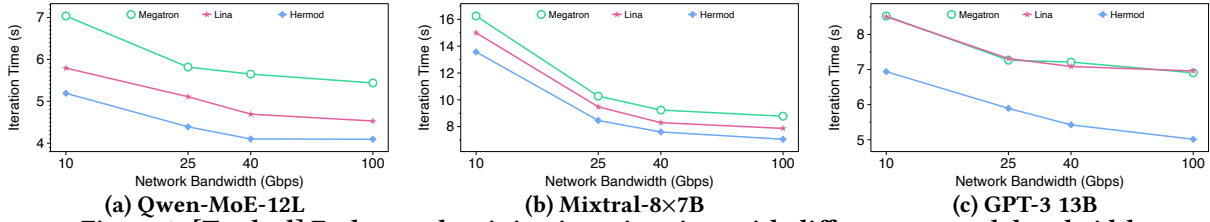
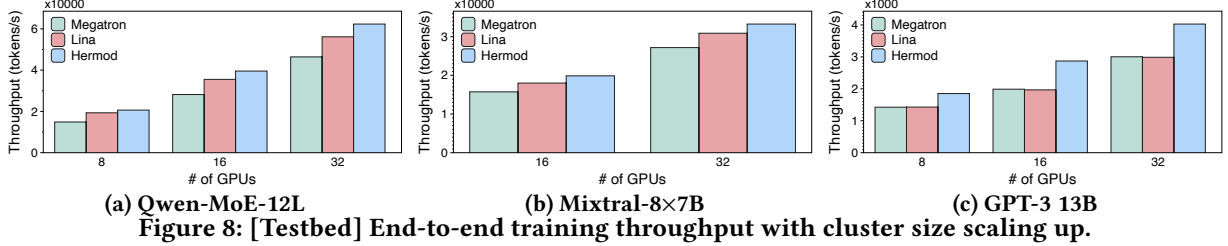


Figure 9: [Testbed] End-to-end training iteration time with different network bandwidth.

for large-size models.

Testbed setup. As shown in Figure 7, our testbed comprises four A100 servers, each equipped with eight NVIDIA A100 GPUs, 96 CPU cores (Intel Xeon Gold 6248R, 3.0 GHz), 503 GB RAM, and two ConnectX-6 100 Gbps NICs. The servers are interconnected by four NVIDIA SN3700 Ethernet switches, forming a full-bisection, rail-optimized network [2]. Within each server, GPUs communicate via NVLink. All servers run Ubuntu 22.04, CUDA 12.4, PyTorch 2.3.1, and NCCL 2.20.5.

Simulation setup. Our simulator is built on top of FlexFlow’s simulator [3] and the packet-level simulator htsim [4], following a similar methodology as [67]. We extend FlexFlow to generate DAGs that describe both computation and communication tasks across all coflow types, which are then fed into htsim to simulate packet communication between GPUs. By default, we simulate a 128-server cluster, each with eight GPUs, connected via a full-bisection fat-tree network with 100Gbps links and a 1μs link propagation delay.

Models and parallelization strategies. We list both the models and default parallelization strategies in Table 5. We consider three representative models: two sparse MoE models, Qwen1.5-MoE [69] and Mixtral-MoE [33], and one dense model, GPT-3 [16]. Moderate model sizes are used for testbed experiments, while larger variants are evaluated in simulation. We vary the DP degree to study scalability. All experiments are conducted using Megatron-LM [61] with pure PyTorch DDP [41].

Baselines. We compare our system against vanilla Megatron-

LM [61] and the state-of-the-art communication scheduler Lina [40]. As Lina is not open-sourced, we re-implemented its core design within Megatron-LM. We exclude conventional coflow schedulers, as they are not tailored for LLM training, and ByteScheduler [52], whose open-source implementation [1] is outdated and incompatible with current NCCL and PyTorch versions.

Metrics. We use # of tokens processed per second and iteration time as metrics to evaluate training performance.

6.2 Testbed Experiments

We conduct testbed experiments to evaluate Hermod’s performance in end-to-end LLM training and to analyze the effectiveness of its individual components.

6.2.1 End-to-end performance Scalability. We first evaluate the scalability of Hermod. We adopt base parallelization configurations (t, p, d, e) of $(1, 2, 1, 4)$, $(1, 2, 1, 8)$, and $(4, 2, 1, 1)$ for Qwen-MoE-12L, Mixtral-8×7B, and GPT-3 13B, respectively, and scale the cluster from 8 to 32 GPUs by increasing the DP degree. For Mixtral-8×7B, we scale only from 16 to 32 GPUs, as the 8-GPU configuration runs out of memory.

Results are shown in Figure 8. Megatron exhibits limited scalability across all models because it ignores inter-coflow relationships. For instance, when scaling GPT-3 13B from 8 to 16 GPUs, Megatron achieves only a 1.39× speedup despite doubling cluster size. Lina improves performance for MoE models but remains ineffective for GPT-3 13B, as it exclusively optimizes EP coflows and provides no benefit

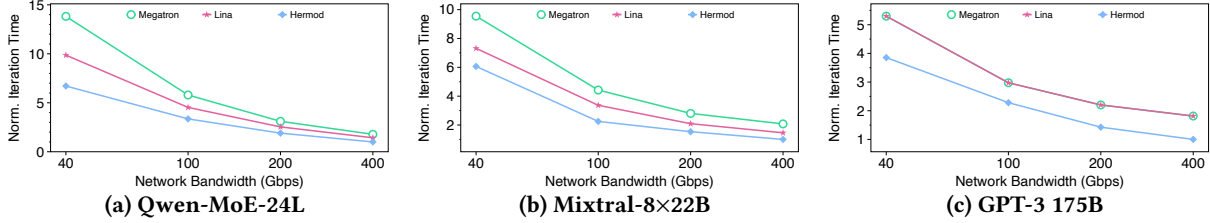
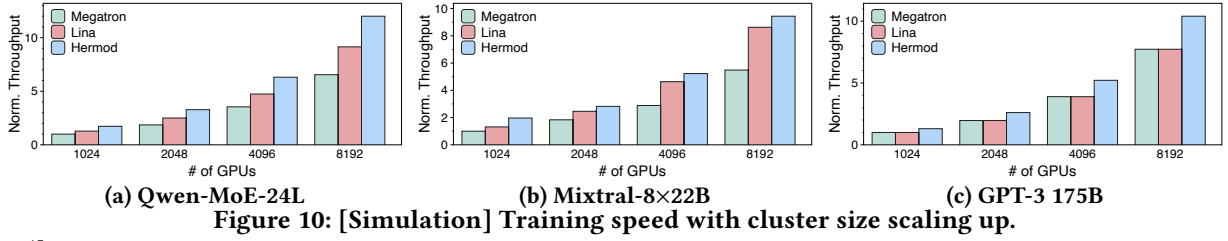


Figure 11: [Simulation] Training speed in the cluster of 128 servers with 1024 GPUs with different bandwidths.

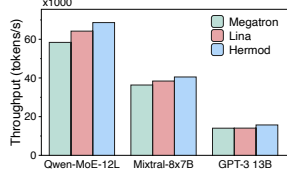


Figure 12: [Testbed] Effectiveness of Hermod in interleaved 1F1B.

for dense models. In contrast, Hermod consistently outperforms both baselines across all workloads, with particularly strong gains on GPT-3 13B, achieving up to 1.44 $\times$ higher throughput. These improvements stem from Hermod’s cluster-wide coflow-aware scheduling and line-rate transmission for highest-priority coflows.

Different network bandwidths. We next evaluate the performance gains of Hermod over the baselines under varying network bandwidths, ranging from 10 Gbps to 100 Gbps. Experiments use the parallelization strategies in Table 5 on a 32-GPU cluster, with available bandwidth controlled by rate-limiting Mellanox NICs via `mlnx_qos`.

The results are shown in Figure 9. Hermod consistently outperforms the baselines across all models and network bandwidths. For instance, when training Qwen-MoE-12L, both Hermod and Lina achieve higher throughput than Megatron, primarily due to the large volume of EP coflows (>80% of total communication). Hermod further surpasses Lina by up to 1.2 $\times$, benefiting from cluster-wide prioritization of earlier microbatch and PP coflows over DP coflows. For GPT-3 13B at 100 Gbps, Hermod achieves up to 1.38 $\times$ higher throughput than the baselines. This improvement is attributed to the absence of heavy EP coflows, which amplifies the interference between PP and DP coflows. As a result, Hermod fully utilizes the network bandwidth through inter-coflow scheduling with strict priority and line-rate transmission, optimizing the global JCT.

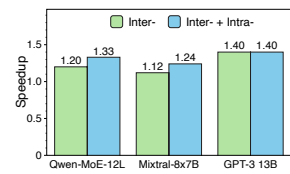


Figure 13: [Testbed] Effectiveness of Hermod’s components.

6.2.2 Hermod Deep Dive We conduct deep dive experiments to evaluate both the efficiency of Hermod in interleaved 1F1B scenario and the effectiveness of its components.

Interleaved 1F1B scenario. We evaluate the effectiveness of Hermod in the complex interleaved 1F1B scenario using the priority illustrated in §4.1 and the default parallelization strategies in the 32-GPU cluster. The results in Figure 12 show similar performance improvements as §6.2.1: Hermod outperforms Megatron and Lina by 1.18 $\times$ and 1.12 $\times$, respectively, demonstrating that Hermod enhances training performance even in the intricate interleaved scenario.

Effectiveness of components. We further investigate the individual contributions of Hermod’s components, specifically inter- and intra-coflow scheduling, in the 32-GPU cluster with default parallelisms as well. The results in Figure 13 reveal that inter-coflow scheduling contributes most to the average performance improvement by reducing cross-coflow interference. With inter-coflow only, the speedups for training Qwen-MoE-12L, Mixtral-8x7B, and GPT-3 13B already reach 1.20 $\times$, 1.12 $\times$, and 1.40 $\times$, respectively, as it significantly reduces interference between coflows at the cluster level. Intra-coflow scheduling targets imbalanced EP coflows and incast-induced stragglers, and thus provides additional benefits for MoE models, while offering no improvement for GPT-3 13B, whose coflows are balanced and already optimally served by line-rate transmission (§4.2.1). This is consistent with our design goal, as intra-coflow scheduling is not intended to optimize balanced cases but to ensure robust performance for imbalanced EP coflows. With both components enabled, Hermod achieves overall speedups of 1.33 $\times$, 1.24 $\times$, and 1.40 $\times$, respectively.

6.3 Large-scale Simulation Experiments

We conduct large-scale simulations to evaluate Hermod when training large models on large clusters.

Scalability. We evaluate scalability by scaling the cluster

size from 1024 to 8192 GPUs with increasing DP degree. The results in Figure 10 show consistent results as the testbed experiments (§6.2.1), Megatron scales poorly across all models, while Lina improves performance for MoE models but remains ineffective for GPT-3 175B. In contrast, Hermod scales well with the cluster size, outperforming the baselines by up to 1.50 $\times$. When training with 8192 GPUs, Hermod achieves 83% and 31% higher throughput compared to Megatron and Lina, respectively. As the cluster size increases from 1024 to 8192 GPUs, the speedup becomes more pronounced. We anticipate that Hermod's performance improvements will be even more significant when training larger models on larger clusters.

Different network bandwidths. We next evaluate Hermod under varying network bandwidths, from 40 Gbps to 400 Gbps, using the parallelization strategies in Table 5 on a 1024-GPU cluster. Results in Figure 11 show that Hermod consistently outperforms the baselines across all bandwidths. Specifically, when bandwidth is limited, such as at 40Gbps, Hermod achieves up to 2.06 $\times$ speedup over Megatron due to its priority scheduling at both endhosts and switches. When bandwidth is sufficient (e.g., 400 Gbps), the improvement of Hermod is less pronounced as the network is no longer the bottleneck. However, Hermod still outperforms the baselines by up to 1.81 $\times$ by reducing EP/PP blocking and overlapping DP coflows.

7 Discussion

Coflows with multiple paths. Similar to [13, 21, 23], we assume that flow routing within and across coflows is determined by the network layer, utilizing mechanisms such as ECMP or packet spraying. It is conceivable that the performance could be further enhanced by co-designing routing with coflow scheduling [73]. Developing optimal algorithms for such joint design is an interesting future direction.

Multi-job coflow scheduling. As highlighted in §3.1, Hermod focuses on coflow scheduling within a single LLM training job, which is a common practice in modern training clusters [35]. In clusters running multiple concurrent jobs, Hermod can be naturally composed with an existing job-level scheduler by assigning each job a weight and using it to scale the priorities of its coflows. In this sense, Hermod addresses a job-internal scheduling problem that is orthogonal to multi-job arbitration, and can be directly integrated into a hierarchical scheduler.

Generalization to other schedules & patterns. Hermod does not rely on hard-coded priority rules. Instead, it derives coflow priorities by extracting critical paths and coflow consumption times from the training DAG (§4.1), and characterizes intra-coflow behavior based on flow distributions (§4.2). Therefore, when the training DAG or execution schedule changes, the priorities are recomputed accordingly. The adap-

tation of Hermod to the interleaved 1F1B schedule exemplifies this generality (§4.1.3).

Failure handling. Network failures are prevalent during training and can significantly impact job performance [28, 30, 44, 53, 61, 63, 75]. Existing systems typically rely on diagnosis tools [35, 44] to detect failures and trigger recovery actions, such as re-routing affected communications, often at the cost of increased JCT. Hermod is complementary to these mechanisms and can directly benefit from their outputs by re-scheduling the remaining coflows accordingly.

Relationship with network transport. Hermod is implemented on top of transport layer with DiffServ support [10] for cluster-wide priority scheduling. Our current prototype is built on RoCEv2 (elaborated in §5); however, the design of Hermod only relies on generic priority and congestion control abstractions, and is not specific to any particular transport. Therefore, it can be naturally applied to other transports such as InfiniBand [11] and MLT [65].

8 Related Work

In addition to the related works mentioned in §2.3, we also summarize others in the following domains.

General coflow schedulers. General coflow schedulers [13, 20–23, 25, 72] focus on optimizing the average CCT or satisfying coflow deadlines. While these schedulers address general coflow scheduling, they do not account for dependencies between coflows derived from LLM training, which can significantly impact end-to-end training performance.

Joint optimizations with scheduling for training. SYNDICATE [46] accelerates DLRM training by co-optimizing collectives and execution across heterogeneous interconnects. Similarly, synthesizers [17, 19, 45, 58] automatically generate efficient collective schedules for diverse topologies. These works primarily target collective optimization in heterogeneous networks and are complementary to Hermod.

AI-specific Networks. Various AI-specific networks have emerged. Some improve communication efficiency via novel topologies [12, 36, 42, 62, 67, 68], while others enhance resiliency for fault tolerance [28, 35, 53]. Integrating Hermod into these systems could further accelerate LLM training.

9 Conclusion

We present Hermod, a near-optimal coflow scheduler that orchestrates all coflow types at the cluster level during LLM training. Hermod employs the insight that coflows are characterized by three model factors, and designs (1) model factor-driven inter-coflow scheduling to align with the LLM training DAG, and (2) matching-based intra-coflow scheduling to maximize transmission rates for high-priority coflows. Extensive evaluations validate that Hermod achieves its design goals. We believe our revisit of coflow scheduling in LLM training will inspire future LLM training optimizations.

This work does not raise any ethical issues.

References

- [1] Bytescheduler source code. <https://github.com/bytedance/byteps/tree/bytescheduler>, 2024.
- [2] Doubling all2all performance with nvidia collective communication library 2.12. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>, 2024.
- [3] Flexflow source code. <https://github.com/flexflow/FlexFlow>, 2024.
- [4] htsim packet-level simulator. <https://github.com/nets-cs-pub-ro/NDP/wiki/NDP-Simulator>, 2024.
- [5] Massively scale your deep learning training with nccl 2.4. <https://developer.nvidia.com/blog/massively-scale-deep-learning-training-nccl-2-4/>, 2024.
- [6] Megatron-lm moe token dispatcher. https://github.com/NVIDIA/Megatron-LM/blob/main/megatron/core/transformer/moe/token_dispatcher.py#L386, 2024.
- [7] Nvidia collective communication library. <https://developer.nvidia.com/nccl>, 2024.
- [8] Nvidia dgx superpod. <https://www.nvidia.com/en-us/data-center/dgx-superpod/>, 2024.
- [9] Supporting overlapping collective communication for data parallel. <https://github.com/NVIDIA/Megatron-LM/issues/391>, 2024.
- [10] Configuration guidelines for diffserv service classes. <https://datatracker.ietf.org/doc/html/rfc4594>, 2025.
- [11] Introduction to infiniband. https://network.nvidia.com/pdf/whitepapers/IB_Intro_WP_190.pdf, 2025.
- [12] Dennis Abts, Garrin Kimmell, Andrew Ling, John Kim, Matt Boyd, Andrew Bitar, Sahil Parmar, Ibrahim Ahmed, Roberto DiCecco, David Han, et al. A software-defined tensor streaming multiprocessor for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 567–580, 2022.
- [13] Saksham Agarwal, Shijin Rajakrishnan, Akshay Narayan, Rachit Agarwal, David Shmoys, and Amin Vahdat. Sincronia: Near-optimal network design for coflows. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 16–29, 2018.
- [14] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. pfabric: Minimal near-optimal datacenter transport. *ACM SIGCOMM Computer Communication Review*, 43(4):435–446, 2013.
- [15] Tommaso Bonato, Abdul Kabbani, Ahmad Ghalayini, Mohammad Dohadwala, Michael Papamichael, Daniele De Sensi, and Torsten Hoefler. Arcane: Adaptive routing with caching and network exploration, 2024.
- [16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [17] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 62–75, 2021.
- [18] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 1–15, New York, NY, USA, 2024. Association for Computing Machinery.
- [19] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, et al. Syccel: Exploiting symmetry for efficient collective communication scheduling. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 645–662, 2025.
- [20] Mosharaf Chowdhury and Ion Stoica. Coflow: A networking abstraction for cluster applications. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks*, pages 31–36, 2012.
- [21] Mosharaf Chowdhury and Ion Stoica. Efficient coflow scheduling without prior knowledge. *ACM SIGCOMM Computer Communication Review*, 45(4):393–406, 2015.
- [22] Mosharaf Chowdhury, Matei Zaharia, Justin Ma, Michael I Jordan, and Ion Stoica. Managing data transfers in computer clusters with orchestra. *ACM SIGCOMM computer communication review*, 41(4):98–109, 2011.
- [23] Mosharaf Chowdhury, Yuan Zhong, and Ion Stoica. Efficient coflow scheduling with varys. In *Proceedings of the 2014 ACM conference on SIGCOMM*, pages 443–454, 2014.
- [24] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [25] Fahad R Dogar, Thomas Karagiannis, Hitesh Ballani, and Antony Rowstron. Decentralized task-aware scheduling for data center networks. *ACM SIGCOMM Computer Communication Review*, 44(4):431–442, 2014.
- [26] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [27] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- [28] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, 2024.
- [29] Sayed Hadi Hashemi, Sangeetha Abdu Jyothi, and Roy Campbell. Tictac: Accelerating distributed deep learning with communication scheduling. *Proceedings of Machine Learning and Systems*, 1:418–430, 2019.
- [30] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, et al. Characterization of large language model development in the datacenter. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 709–729, 2024.
- [31] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [32] Anand Jayarajan, Jinliang Wei, Garth Gibson, Alexandra Fedorova, and Gennady Pekhimenko. Priority-based parameter propagation for distributed dnn training. *Proceedings of Machine Learning and Systems*, 1:132–145, 2019.
- [33] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [34] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. A unified architecture for accelerating distributed {DNN} training in heterogeneous {GPU/CPU} clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 463–479, 2020.

- [35] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibao Nong, et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024.
- [36] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14, 2023.
- [37] James E Kelley. The critical-path method: resource planning and scheduling. *Industrial scheduling*, 1963.
- [38] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5:341–353, 2023.
- [39] Harold W Kuhn. The hungarian method for the assignment problem. *Naval research logistics quarterly*, 2(1-2):83–97, 1955.
- [40] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 945–959, 2023.
- [41] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704*, 2020.
- [42] Xudong Liao, Yijun Sun, Han Tian, Xinchun Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, et al. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 554–574, 2025.
- [43] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 486–498, 2023.
- [44] Kefei Liu, Zhuo Jiang, Jiao Zhang, Shixian Guo, Xuan Zhang, Yangyang Bai, Yongbin Dong, Feng Luo, Zhang Zhang, Lei Wang, et al. R-pingmesh: A service-aware roce network monitoring and diagnostic system. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 554–567, 2024.
- [45] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 16–37, 2024.
- [46] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. Better together: Jointly optimizing {ML} collective scheduling and execution planning using {SYNDICATE}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, 2023.
- [47] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Se-shadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM symposium on operating systems principles*, pages 1–15, 2019.
- [48] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.
- [49] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.
- [50] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [51] Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.
- [52] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 16–29, 2019.
- [53] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 691–706, 2024.
- [54] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [55] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- [56] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.
- [57] Mehrnoosh Shafiee and Javad Ghaderi. Scheduling coflows with dependency graph. *IEEE/ACM Transactions on Networking*, 30(1):450–463, 2021.
- [58] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, 2023.
- [59] Vinay Shankarkumar, Laurent Montini, Tim Frost, and Greg Dowd. Precision time protocol version 2 (ptpv2) management information base. Technical report, 2017.
- [60] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [61] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [62] Chenchen Shou, Guyue Liu, Hao Nie, Huaiyu Meng, Yu Zhou, Yimin Jiang, Wenqing Lv, Yelong Xu, Yuanwei Lu, Zhang Chen, et al. In-finitehbd: Building datacenter-scale high-bandwidth domain for llm with optical circuit switching transceivers. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 1–23, 2025.
- [63] Borui Wan, Mingji Han, Yiyao Sheng, Zhichao Lai, Mofan Zhang, Junda Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Bytecheckpoint: A unified checkpointing system for llm development. *arXiv preprint arXiv:2407.20143*, 2024.
- [64] Xinchun Wan, Hong Zhang, Hao Wang, Shuihui Hu, Junxue Zhang, and Kai Chen. Rat-resilient allreduce tree for distributed machine learning. In *Proceedings of the 4th Asia-Pacific Workshop on Network-*

- ing, pages 52–57, 2020.
- [65] Hao Wang, Han Tian, Jingrong Chen, Xinchun Wan, Jiacheng Xia, Gaoxiong Zeng, Wei Bai, Junchen Jiang, Yong Wang, and Kai Chen. Towards {Domain-Specific} network transport for distributed {DNN} training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1421–1443, 2024.
 - [66] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. Rail-only: A low-cost high-performance network for training llms with trillion parameters.
 - [67] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. {TopoOpt}: Co-optimizing network topology and parallelization strategy for distributed training jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 739–767, 2023.
 - [68] Zihan Yan, Dan Li, Li Chen, Dian Xiong, Kaihui Gao, Yiwei Zhang, Rui Yan, Menglei Zhang, Bochun Zhang, Zhuo Jiang, et al. From atop to zcube: Automated topology optimization pipeline and a highly cost-effective network topology for large model training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 861–881, 2025.
 - [69] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
 - [70] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *2nd USENIX workshop on hot topics in cloud computing (HotCloud 10)*, 2010.
 - [71] Hao Zhang, Zeyu Zheng, Shizhen Xu, Wei Dai, Qirong Ho, Xiaodan Liang, Zhiting Hu, Jinliang Wei, Pengtao Xie, and Eric P Xing. Poseidon: An efficient communication architecture for distributed deep learning on {GPU} clusters. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 181–193, 2017.
 - [72] Hong Zhang, Li Chen, Bairen Yi, Kai Chen, Mosharaf Chowdhury, and Yanhui Geng. Coda: Toward automatically identifying and scheduling coflows in the dark. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 160–173, 2016.
 - [73] Yangming Zhao, Kai Chen, Wei Bai, Minlan Yu, Chen Tian, Yanhui Geng, Yiming Zhang, Dan Li, and Sheng Wang. Rapier: Integrating routing and scheduling for coflow-aware data center networks. In *2015 IEEE Conference on Computer Communications (INFOCOM)*, pages 424–432. IEEE, 2015.
 - [74] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
 - [75] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, et al. Resiliency at scale: Managing {Google’s}{TPUv4} machine learning supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 761–774, 2024.